

The F2Z verifier for the mod- q opening, with Ligerito as a black box

The `f2z-pcs` repository

2026-07-16

Abstract

This note specifies, at code-review granularity, the verifier of the F2Z polynomial commitment scheme for its core statement — the mod- q integer-MLE evaluation $\text{INT}(D)(r) = y \in \mathbb{F}_q$ over an $\mathbb{F}_2$ -committed bit matrix — as implemented by `verify_mle_eval_mod_q_ligerito` in `src/ligerito_flock.rs`. The flock Ligerito proximity verifier is treated throughout as a correct black box with a one-line interface contract (Definition 1); its internals are explicitly out of scope. The note walks the soundness chain in verification order — committed bits $\rightarrow$ (opening) residual bit-MLE claims $\rightarrow$ (forest GKR) roots $\alpha^u \rightarrow$ (generator binding) integers $u \rightarrow$ (chunk recombination) $y \in \mathbb{F}_q$ — and for every verifier-side quantity states whether it is *recomputed from public data*, *bound by a transcript absorb*, or *oracle-certified*. A reader holding this note and the source should be able to confirm, function by function, that every stated check exists and that no stated check is silently missing.

Contents

1	Scope, review contract, and the oracle	2
2	Statement, instance shape, and the proof object	4
2.1	Instance shape and the committed matrix	4
2.2	The claim and the chunk decomposition	4
2.3	The proof object	5
2.4	Serialization: the host proof stream	5
3	Verification order and the Fiat–Shamir schedule	6
3.1	Transcript primitives	6
3.2	The verifier, step by step	7
3.3	The Fiat–Shamir invariant	9
4	The merged-forest GKR verifier	9
5	The exit-claim linkage: pre-sumcheck, ring switch, η-batching, one oracle call	11
5.1	From the exit claim to a bit-MLE claim: the pre-sumcheck	11
5.2	From L bit-MLE claims to one oracle call: the ring switch	12
5.3	Provenance of every verifier-side quantity	13
6	Integer binding in the clear	13

7	Config derivation and the audited regime	15
8	What the tests pin	15
9	Trust boundary and caveats	16

1 Scope, review contract, and the oracle

F2Z¹ proves an *integer* multilinear-extension evaluation over data whose *bits* are committed with a characteristic-2 commitment. The integer row-fold is carried in the exponent of $\mathbb{K} = \text{GF}(2^{128})$ and certified by a GKR grand-product forest; the only opener is the flock-backed ring-switch + recursive Ligerito pipeline.

What this note covers. Exactly one entry point and its callees:

Function(s)	File
<code>verify_mle_eval_mod_q_ligerito</code>	<code>src/ligerito_flock.rs</code>
<code>verify_int_eval_merged_common</code>	<code>src/ligerito.rs</code>
<code>verify_merged_forest, mle_at, absorb_gfs</code>	<code>src/merged_forest.rs</code>
<code>MLSumcheck::verify_as_subprotocol</code>	<code>src/piop/sumcheck.rs</code>
<code>MultiDegreeSumcheck::verify_as_subprotocol</code>	<code>src/piop/sumcheck/multi_degree.rs</code>
<code>is_generator, mod_q_chunk_width, mod_q_num_chunks, chunk_row_weights, row_bit_weights, recombine_read_off, FixedBasePow</code>	<code>src/pcs.rs</code>
<code>ring-switch pieces: absorb_sv, transpose_bits_128, residual_b_evals</code>	<code>src/ligerito.rs</code>
<code>IntEvalRsLigModQProof::to_bytes / from_bytes</code>	<code>src/ligerito_flock.rs, src/proof_codec.rs</code>
<code>Blake3Transcript, absorb_slice, get_field_challenge</code>	<code>src/transcript/</code>
<code>ZincChallenger</code> (the flock challenger bridge)	<code>src/ligerito_flock.rs</code>

Line numbers drift; navigate by function name.

What this note does not cover.

- *The Ligerito / ring-switch proximity verifier internals (flock-core)*. It appears only as the oracle of Definition 1. Whenever the text says “the opening is checked,” that is the oracle call, full stop.
- The batched multi-polynomial variants (`verify_rs_ligerito_batch`) and the virtual-XOR variants (`verify_mle_eval_mod_q_ligerito_with_virtual_xors, ...claims_only`). They exist as public API and are discussed only in the one paragraph of Section 9 that states their extra trust obligations.
- The prover, the commit path, and all performance machinery (lazy forest schedules, NEON kernels). These are semantics-preserving and pinned byte-identical by tests (Section 8); the verifier never depends on which schedule produced a proof.

¹Repository `f2z-pcs`; a standalone extraction of the most-optimized `f2-int` pipeline from the `zinc-plus` repository (branch `f2-int-unified-ligerito`, snapshot 2026-07). The construction is due to Lev Soukhanov (the `char2-fieldswitch` note, §9); the as-deployed long-form exposition lives in `zinc-plus`’s `documentation/f2-int-eval-doc/`. This note describes the extracted code, which has drifted from the deployment; where the two disagree, this repository’s code is authoritative for this note.

The oracle. The single point of contact with `flock-core` on this path is the following function, used once per verification:

Definition 1 (Ligerito opening oracle). $\text{LigVerify}(vc, \pi_{\text{lig}}, m_p, \beta, \text{root}, \text{EvalB}, \mathcal{C}) \in \{\text{accept}, \text{reject}\}$ — in code, `flock_core::pcs::ligerito::recursive_verifier_with_basis_succinct`. Inputs: a verifier config vc (derived from the instance shape, Section 7); the flock proof object π_{lig} ; the packed variable count m_p ; a target value $\beta \in \mathbb{K}$; the Merkle commitment root ; a callback EvalB which, given a prefix $(\rho_1, \dots, \rho_k) \in \mathbb{K}^k$ and a tail width k' with $k + k' = m_p$, returns the $2^{k'}$ values $\tilde{B}(\rho_1, \dots, \rho_k, \text{bits}(y))$ of a weight function $B: \{0, 1\}^{m_p} \rightarrow \mathbb{K}$ known to the verifier; and a Fiat–Shamir challenger $\mathcal{C}$ that continues the caller’s transcript. Assumed contract: if the oracle accepts, then — except with negligible probability over the challenger’s randomness — the vector $P \in \mathbb{K}^{2^{m_p}}$ committed under root (with the packing of Section 2) satisfies

$$\sum_{y \in \{0,1\}^{m_p}} P[y] \cdot B(y) = \beta.$$

Completeness holds for honestly generated proofs. Everything else — the proximity test, Merkle verification, query counts, grinding, out-of-domain sampling, and the oracle’s own transcript sub-schedule — is inside the box.

Note the contract is an *inner-product* (weighted-sum) opening. The familiar point-evaluation form is the special case where B is an `eq`-derived tensor; the `mod-q` verifier hands the oracle an η -batched combination of L such functionals (Section 5), so the inner-product form is the honest interface.

The soundness chain. `docs/DESIGN.md` states it; the verifier walks it in this order, one section per link:

$$\begin{array}{c} \underbrace{\text{committed bits}}_{\text{oracle, §5}} \rightarrow \underbrace{\text{residual bit-MLE claims } \widetilde{M}(pt_l) = \mu_l}_{\text{ring switch, §5}} \rightarrow \underbrace{\text{exit claims } e_d^{(l)}}_{\text{pre-sumcheck, §5}} \rightarrow \\ \underbrace{\text{roots } \alpha^u}_{\text{forest, §4}} \rightarrow \underbrace{\text{integers } u}_{\text{binding, §6}} \rightarrow \underbrace{y \in \mathbb{F}_q}_{\text{read-off, §6}} \end{array}$$

The verifier *executes* the chain left-to-right per chunk in a different operational order (range/binding checks come first, Section 3); the chain above is the direction in which trust flows once all checks pass.

The computing-vs-binding discipline. A verifier that merely *recomputes* a value from prover-chosen inputs binds nothing. Throughout, every verifier-side quantity is classified as one of:

- (R) *recomputed* from statement data (public inputs the caller vouches for) and verifier randomness;
- (A) prover-supplied and *bound by a transcript absorb* before some later challenge, and/or *checked equal* to an already-bound quantity;
- (O) *oracle-certified* — its correctness is exactly Definition 1’s guarantee.

Table 1 classifies every quantity on the path.

2 Statement, instance shape, and the proof object

2.1 Instance shape and the committed matrix

`IntEvalParams { t, s, word_bits: W }` (`src/pcs.rs`) fixes the shape: the data D is a $2^t \times 2^s$ matrix of W -bit cells, W a power of two, flat index `cell_index(b, c) = (b << s) | c`. Derived quantities used throughout:

$$d = t + \log_2 W, \quad m_p = d - 7 + s, \quad m = m_p + 7 = d + s$$

(code: `row_bit_vars` and `packed_vars`). The *committed object* is the single-bit matrix $M \in \mathbb{F}_2^{2^s \times 2^d}$ with

$$M[c][i] = \text{bit } j \text{ of } D(b, c), \quad i = (b \ll \log_2 W) | j,$$

i.e. row c concatenates the bits of column c 's cells, cell-major, bit-minor (`repack_leaf_bits`). Its multilinear extension $\widetilde{M}: \mathbb{K}^{d+s} \rightarrow \mathbb{K}$ places the d row-bit coordinates *low* and the s column coordinates *high*, LSB-first within each block; all eq tables in the crate follow the same LSB-first convention (`build_eq_x_r_vec`).

For the commitment, the bits are packed 128 per $\mathbb{K}$ -element along the low 7 row-bit coordinates (`LOG_PACKING = 7`): the packed vector $P \in \mathbb{K}^{2^{m_p}}$ has

$$P[c \cdot 2^{d-7} + i_{\text{hi}}] = \sum_{v=0}^{127} M[c][(i_{\text{hi}} \ll 7) | v] \cdot \beta_v,$$

where β_v is the v -th bit of the $\mathbb{K}$ -element representation (monomial basis, LSB-first; `commit_rs_flock_from_rows`). P is what flock commits: `flock_core::pcs::commit::commit(&p_msg, &PcsParams { m: m_p + 7, log_inv_rate, log_batch_size, profile })` — the RS encoding, interleaving and Merkle tree are inside the oracle's box; the verifier only ever touches the `Commitment`'s root and shape parameters. *Precondition*: the commitment handed to the verifier was produced at this same shape; the verifier does not cross-check `commitment.params` against `p` (a mismatch is left to the oracle to reject).

2.2 The claim and the chunk decomposition

The verifier's signature is

```
verify_mle_eval_mod_q_ligerito<R>(transcript, commitment, proof, p, row_weights_q, col_weights, alpha, claimed, q_bits, vc)
```

with R any ring of characteristic $\neq 2$ exposing `From<u128>`, $+$, $\times$. The relation verified is, for the caller-supplied weight vectors $w \in [0, 2^{q_{\text{bits}}}]^{2^t}$ (`row_weights_q`, as `u128s`) and $w' \in R^{2^s}$ (`col_weights`):

$$y = \sum_{c \in \{0,1\}^s} w'_c \sum_{b \in \{0,1\}^t} w_b \cdot \text{INT}(D(b, c)) \quad \text{in } R, \tag{1}$$

where $\text{INT}(D(b, c)) = \sum_j 2^j M[c][(b \ll \log_2 W) | j]$ is the cell's integer value. The *intended instantiation* is the genuine MLE opening: $R = \mathbb{F}_q$, $w_b = \text{eq}(b, r_1) \bmod q \in [0, q)$ the canonical integer representatives, $w'_c = \text{eq}(c, r_2) \in \mathbb{F}_q$, whence $y = \widetilde{\text{INT}}(D)(r_1, r_2)$. The verifier itself is agnostic: any weights satisfying the width precondition are accepted, and soundness is with respect to (1) for the weights given.

Full-width row weights ($\approx q_{\text{bits}} \approx 100$ bits) would overflow the exponent’s injectivity budget, so they are chunked (`src/pcs.rs`):

$$c_w = 127 - t - W \quad (\text{mod_q_chunk_width; asserts } t + W \leq 126),$$

$$L = \max(\lceil q_{\text{bits}}/c_w \rceil, 1) \quad (\text{mod_q_num_chunks}),$$

$$W_b^{(l)} = (w_b \gg c_w l) \wedge (2^{c_w} - 1) \quad (\text{chunk_row_weights}), \quad l = 0, \dots, L - 1,$$

so $w_b = \sum_l W_b^{(l)} 2^{c_w l}$ *exactly* whenever $w_b < 2^{c_w L}$ — guaranteed by the precondition $w_b < 2^{q_{\text{bits}}} \leq 2^{c_w L}$. (Weights at or above $2^{c_w L}$ would be silently truncated identically on both sides; the verified relation is always (1) with w_b replaced by $\sum_l W_b^{(l)} 2^{c_w l}$.) Each chunk’s honest fold

$$u_c^{(l)} = \sum_b W_b^{(l)} \cdot \text{INT}(D(b, c)) \leq (2^{c_w} - 1)(2^W - 1) 2^t < 2^{c_w + t + W} = 2^{127} \quad (2)$$

fits below the generator order and binds injectively in the exponent (Section 6). Prover and verifier derive $(c_w, L, W^{(l)})$ from the same public data; nothing about the chunking rides the proof.

2.3 The proof object

`IntEvalRsLigModQProof (src/ligerito_flock.rs)`:

Field	Role
<code>mfs:</code> <code>Vec<MergedForestProof></code>	One merged-forest proof per chunk (L total). Each is a <code>Vec<MergedLayer></code> of d layers; a <code>MergedLayer</code> holds an optional phase-A sumcheck proof <code>sc_x</code> (absent at layer 0), the phase-B sumcheck proof <code>sc_c</code> , and the closing child pair <code>pair</code> $\in \mathbb{K}^2$. <i>The per-tree roots are not proof data</i> : the verifier derives them from <code>us</code> (Section 4).
<code>us:</code> <code>Vec<Vec<u128>></code>	<code>us[1]</code> = $(u_c^{(l)})_{c \in \{0,1\}^s}$, the 2^s sent chunk folds of chunk l ; claimed to equal (2).
<code>presums:</code> <code>Vec<MultiDegreeSumcheckProof></code>	One de-black-boxing pre-sumcheck per chunk: d rounds, one group, degree 2 when honest (the degree field rides the proof; Remark 1), plus the group’s claimed sum.
<code>rings:</code> <code>Vec<RingSwitchProof></code>	One ring-switch message per chunk: <code>s_v</code> , the 128 partial evaluations $s_v^{(l)} = \widetilde{M}_v(pt_l^{\text{hi}})$ (Section 5).
<code>lig:</code> <code>LigeritoProof</code>	flock’s serde proof object for the single batched opening — opaque here (Definition 1).

2.4 Serialization: the host proof stream

`to_bytes / from_bytes (src/ligerito_flock.rs, codec primitives in src/proof_codec.rs)`. All scalars are little-endian. The stream grammar is, in order:

Bytes	Content
u64	L (chunk count)
<i>per chunk l:</i>	
u64	layer count ($= d$ when honest)
<i>per layer:</i> u64	sc_x presence flag (0 or 1)
[Transcribable]	sc_x (iff flag = 1)
Transcribable	sc_c
2×16 B	closing pair (p, q) , each a $\mathbb{K}$ -element (two LE words)
u64 + $n_u \times 16$ B	fold count n_u , then us [1] as LE u128s
Transcribable	presums [1]
u64 + $n_{sv} \times 16$ B	ring length n_{sv} ($= 128$ when honest), then s_v
u64 + n B	length-prefixed bincode 1.3 blob of lig (flock’s own pinned encoder)

Transcribable values are self-describing: a **u32** LE byte length, then the payload. A **SumcheckProof** payload is, in order: a 16-byte field-modulus tag (the **0x87** pattern); a **u32** message count; per message, a **u32** evaluation count followed by that many 16-byte evaluations; and the 16-byte claimed sum. A **MultiDegreeSumcheckProof** payload is: the 16-byte modulus tag; a **u32** group count; a **u32** round count; one **u32** degree per group; then all groups’ round messages (per group, per round, the degree-many 16-byte evaluations); then one 16-byte claimed sum per group.

What from_bytes rejects, exactly. The outer reader returns **CodecError::Truncated** whenever a **u64**/**u128**/ $\mathbb{K}$ -element/blob read would run past the buffer (including overflow of the cursor), and **CodecError::Bincode** when the embedded **LigeritoProof** blob fails bincode decoding. Everything else is deferred to verification: counts inconsistent with the statement ($L \neq \lceil q_{\text{bits}}/c_w \rceil$, wrong layer counts, wrong ring lengths, ...) decode fine and are rejected by the shape checks of Section 3.

Codec caveats (robustness, not soundness). Three properties a reviewer should know; none creates a false-acceptance path, since every decoded proof still faces the full verifier:

- *Panic paths on malformed streams.* The **Transcribable** sub-decoders are **assert! / expect-**based: a truncated or length-inconsistent inner payload, or an embedded field-modulus tag $\neq \mathbf{0x87}$, aborts by panic rather than returning **CodecError**. A host feeding untrusted bytes should treat **from_bytes** as panicking-on-hostile-input (e.g. wrap it) until hardened.
- *Non-strict decoding.* Trailing bytes after the **lig** blob are ignored (no end-of-stream check), and any **sc_x** presence flag $\neq 1$ decodes as “absent.” Hence distinct byte streams can decode to one proof; canonicity (*re*-serialization being byte-identical, pinned by test) is a property of **to_bytes** output, not of arbitrary accepted input.
- *No semantic validation at decode.* By design: the codec is a framing layer; the verifier is the validator.

3 Verification order and the Fiat–Shamir schedule

3.1 Transcript primitives

The transcript is a Blake3 hasher (**Blake3Transcript**, **src/transcript/mod.rs**). Byte-level semantics:

- `absorb_slice(buf)` updates the hasher with `0x06 || buf || 0x07`.
- `absorb_gfs(tag, vals)` (`src/merged_forest.rs`) and `absorb_gf_slice(tag, vals)` (`src/ligerito.rs`) call `absorb_slice` on one domain-tag byte followed by each $\mathbb{K}$ -element as two LE u64 words. Tags used on this path: `0x30` (forest roots), `0x32` (layer closing pair), `0x20` (ring-switch `s_v`). (Tag `0x37`, virtual-XOR externals, does not occur here.)
- `absorb_random_field(v)` — used by the sumcheck layers — updates with

$$0x03 \parallel \text{modulus} \parallel 0x05 \parallel 0x01 \parallel \text{inner} \parallel 0x03,$$

where for $\mathbb{K}$ the modulus bytes are the fixed 16-byte LE pattern of `0x87` (the reduction pentanomial $X^{128} + X^7 + X^2 + X + 1$ with the X^{128} bit implicit) and the inner bytes are the element’s 16-byte LE representation.

- `get_field_challenge::<Gf>()` squeezes 16 bytes from the Blake3 XOF (*without* advancing the state), then ratchets the state with `0x12 || squeezed bytes || 0x34`, and interprets the squeezed bytes as a $\mathbb{K}$ -element (raw 128 bits, LSB-first; every 16-byte string is a valid element, so challenges are uniform over $\mathbb{K}$). `get_field_challenges(n)` is n sequential such calls.
- `ZincChallenger` (`src/ligerito_flock.rs`) drives flock’s `Challenger` trait from the same transcript: `observe_label/observe_bytes/observe_f128(_slice)` map to `absorb_slice` (elements as 16-byte LE), `sample_f128` maps to `get_field_challenge`, and the proof-of-work hooks squeeze one field element as grinding seed and absorb the 8-byte LE nonce via `absorb_slice` (`verify_pow` absorbs the nonce whether or not the grinding check passes, keeping the transcript in lockstep; an honest verifier rejects on failure inside the oracle). Thus the oracle’s sub-schedule *continues the same Fiat–Shamir chain*; its contents are out of scope.

3.2 The verifier, step by step

Below is the complete operational order of `verify_mle_eval_mod_q_ligerito`, with every check anchored `[V·]` and every transcript action marked **A** (absorb) / **S** (squeeze). The prover performs the identical **A/S** sequence at the corresponding stages, so transcripts stay in lockstep; any divergence misaligns every later challenge and the run rejects (with overwhelming probability) at the first challenge-dependent check.

1. *Setup (no transcript activity)*. Recompute c_w, L ; `[V0]` reject `Shape` unless $|\text{mfs}| = |\text{us}| = |\text{presums}| = |\text{rings}| = L$; recompute the chunk weights $W^{(l)}$ from `row_weights_q`; set $\text{bound} = 2^{c_w+t+W} = 2^{127}$.
2. *Per chunk $l = 0, \dots, L - 1$, sequentially:*
 - (a) `[V1]` *Range*: for every c : $u_c^{(l)} < 2^{127}$, else `ChunkRange{chunk: l, col: c}`. (Runs before any of chunk l ’s transcript activity.)
 - (b) Enter `verify_int_eval_merged_common` with weights $W^{(l)}$:
 - i. `[V2]` $|\text{us}[1]| = 2^s$, else `Forest`.
 - ii. `[V3]` `is_generator( $\alpha$ )`, else `ChallengeNotGenerator` (Section 6; evaluated identically every chunk).

- iii. [V4] $\max_c u_c^{(l)} < 2^{128} - 1$, else **Magnitude**. Vacuous given [V1] (kept for shape parity with the unchunked path).
 - iv. *Recompute roots* $R_c^{(l)} = \alpha^{u_c^{(l)}}$ (fixed-base comb, `FixedBasePow::new( $\alpha$ , 128, 8)`). (**R**) from proof data — binding comes from the next step’s absorb.
 - v. Run `verify_merged_forest` (Section 4): [V5]–[V11], absorbs **A**(0x30:roots), **S**($\zeta \in \mathbb{K}^s$), then per layer the phase-A/B sumcheck exchanges and **A**(0x32:pair), **S**(λ_ℓ). Output: exit point $z^{(l)} = (z_x^{(l)}, z_c^{(l)}) \in \mathbb{K}^d \times \mathbb{K}^s$ and exit value $e_d^{(l)}$. Failure maps to **Forest**.
 - vi. Run `MultiDegreeSumcheck::verify_as_subprotocol` on `presums[1]` with round count d (Section 5): absorbs **A**(d), **A**(n_{groups}), **A**(degree) (each via `absorb_random_field`), then per round **A**(round message), **S**(challenge), **A**(challenge); checks [V12] (round count = d) and [V13] (per-round message length = declared degree, enforced at the end). Output: shared point $r^{*(l)} \in \mathbb{K}^d$ and expected evaluation $E^{(l)}$. Failure maps to **PreSumcheck**.
 - vii. [V14] `presums[1].claimed_sums()` = $[e_d^{(l)} + 1]$ (one group, exactly this value), else **PreSumcheck**.
 - viii. Recompute $\hat{R}^{(l)} = \sum_i q_{\text{rowbit}}^{(l)}[i] \cdot \text{eq}(i, r^{*(l)})$ with $q_{\text{rowbit}}^{(l)}[i] = \text{eq}(i, z_x^{(l)}) \cdot (\alpha^{W_b^{(l)} 2^j} - 1)$, $i = (b \ll \log_2 W) | j$ (`row_bit_weights`; the verifier’s one $O(2^d)$ step). [V15] $\hat{R}^{(l)} \neq 0$, else **RHatZero**. Set $\mu_l = E^{(l)} \cdot (\hat{R}^{(l)})^{-1}$ and $pt_l = (r^{*(l)} \parallel z_c^{(l)}) \in \mathbb{K}^{d+s}$.
3. *Ring-switch checks, per chunk* $l = 0, \dots, L - 1$: [V16] $|\mathbf{s}_v^{(l)}| = 128$, else **Shape**; [V17] $\sum_{v=0}^{127} \text{eq}(v, pt_l[0..7]) \cdot s_v^{(l)} = \mu_l$, else **RingSwitchClaim**; then **A**(0x20: $\mathbf{s}_v^{(l)}$). (Check first, absorb second; the prover absorbs unconditionally at the same position.)
 4. **S**($r'' \in \mathbb{K}^7$), then **S**($\eta \in \mathbb{K}^L$) — seven then L sequential field challenges. Build $\text{eq}_{r''} \in \mathbb{K}^{128}$.
 5. *Target*. Recompute $\beta_l = \sum_{u=0}^{127} \text{eq}_{r''}[u] \cdot (\text{transpose}_{128}(\mathbf{s}_v^{(l)}))[u]$ and $\beta = \sum_l \eta_l \beta_l$ (`transpose_bits_128`: the 128×128 bit transpose, $\text{bit}_v(\text{out}[u]) = \text{bit}_u(\text{in}[v])$).
 6. [V18] *The oracle call* (Definition 1):

$$\text{LigVerify}(vc, \text{lig}, m_p, \beta, \text{root}, \text{EvalB}, \mathcal{C}),$$

with `EvalB` the closure over $\{pt_l^{\text{hi}}\}_l = \{pt_l[7..]\}_l$, $\text{eq}_{r''}$ and η described in Section 5, and $\mathcal{C} = \text{ZincChallenger}(\text{transcript})$. Reject **LigeritoReject** if the oracle rejects. All transcript activity inside is the oracle’s.

7. [V19] *Read-off* (after the oracle; pure recomputation):

$$y_{\text{ver}} = \sum_c w'_c \sum_l 2^{c w_l} u_c^{(l)} \quad \text{in } R \quad (\text{recombine_read_off}),$$

reject **ReadOff** unless $y_{\text{ver}} = \text{claimed}$.

Error type: **FlockRsError**; the variants reachable here are **RingSwitch** (of **Shape** or **RingSwitchClaim**), **ChunkRange**, **LigeritoReject**, and **Common** (of **Forest**, **ChallengeNotGenerator**, **Magnitude**, **PreSumcheck**, **RHatZero**, or **ReadOff**).

The verifier is fail-fast: the *first* failed check aborts, so later transcript activity does not occur on rejection (relevant only to error-behavior review, not to soundness).

3.3 The Fiat–Shamir invariant

Every squeeze follows the absorbs of everything that squeeze must bind. Concretely, in transcript order:

- $\zeta^{(l)}$ follows $\mathbf{A}(0\mathbf{x}30)$ of chunk l ’s recomputed roots — this is the absorb that binds the sent folds `us[1]` (the roots are a deterministic function $\alpha^{u_c^{(l)}}$ of them), and transitively the entire earlier transcript, including all previous chunks.
- Each sumcheck round challenge follows the absorb of that round’s message(s); each layer’s line challenge λ_ℓ follows $\mathbf{A}(0\mathbf{x}32)$ of that layer’s closing pair; the pre-sumcheck’s metadata (d , group count, degree) is absorbed before its first round.
- r'' and η follow the absorbs of *all* L ring messages $\mathbf{s.v}^{(l)}$ (and everything before them); the oracle target β is a function only of absorbed data ($\mathbf{s.v}$) and of r'', η themselves.
- The oracle’s internal challenges continue the same chain, hence follow everything above.

Obligation 1 (Statement binding is the caller’s). *Neither `verify_mle_eval_mod_q_ligerito` nor the in-repo tests absorb the commitment root, the weights, q_{bits} , the claimed y , or α into the transcript: the API takes them as inputs, and the module documentation (`src/ligerito.rs`) places the absorb of the root “before any challenge is drawn” with the caller. In a Fiat–Shamir deployment the host must (i) absorb the commitment root and all statement data into this transcript before the first squeeze-bearing call, and (ii) sample any statement randomness (the evaluation point behind the weights; α , if random) from that transcript. For α specifically, binding soundness does not require randomness — any generator works (Section 6) — but it must be fixed before the prover commits to `us`.*

4 The merged-forest GKR verifier

`verify_merged_forest(transcript, roots, proof, depth = d, s)` (`src/merged_forest.rs`) verifies that a family of 2^s product trees of depth d , whose leaf layer is the (never-materialized) multilinear family $\Lambda^{(l)}[c][i] \in \mathbb{K}$, has per-tree roots equal to the supplied `roots`, reducing the statement to a single evaluation claim on the leaf MLE. On this path the leaves are the bit-affine values

$$\Lambda^{(l)}[c][i] = 1 + M[c][i] \cdot \tau^{(l)}(i), \quad \tau^{(l)}(i) = \alpha^{W_b^{(l)2^j}} - 1 \quad (i = (b \ll \log_2 W)|j), \quad (3)$$

so that $\prod_i \Lambda^{(l)}[c][i] = \alpha^{\sum_b W_b^{(l)} \text{INT}(D(b,c))} = \alpha^{u_c^{(l)}}$ for the *true* fold — the whole point of the forest. (In $\mathbb{K}$, $-1 = +1$; we keep the -1 form for readability.)

Roots are derived, not carried. The caller recomputes $R_c = \alpha^{u_c}$ from the sent folds and passes them in; the proof object has no root field. The very first transcript action, $\mathbf{A}(0\mathbf{x}30: R_0, \dots, R_{2^s-1})$, is what binds the folds: a prover whose forest messages were generated against different roots faces challenges derived from *these* roots and fails the layer checks with overwhelming probability. *The absorb is the binding.*

Entry. [V5] Shape unless $|\text{roots}| = 2^s$ and $|\text{layers}| = d$. Then $\mathbf{S}(\zeta \in \mathbb{K}^s)$ and the verifier computes the entry claim *itself*:

$$C_0 = \text{mle_at}(\text{roots}, \zeta) = \tilde{R}(\zeta) \quad (\mathbf{R}),$$

by s successive low-bit binds ($O(2^s)$ work). No prover data is involved in C_0 .

Layer loop. The trees' levels form one multilinear family $L_\ell(x, c)$ over $(x, c) \in \{0, 1\}^\ell \times \{0, 1\}^s$ (x = in-tree index, low; c = tree index, high) with the top-bit product recursion $L_\ell(x, c) = L_{\ell+1}(x, 0, c) \cdot L_{\ell+1}(x, 1, c)$ (the new child bit is the *top* in-tree bit). The verifier maintains a running claim $C_\ell = \widetilde{L}_\ell(z_x, z_c)$, starting from $(z_x, z_c) = (\varepsilon, \zeta)$, C_0 as above. For each layer $\ell = 0, \dots, d-1$, the claim

$$C_\ell = \sum_{x \in \{0,1\}^\ell} \sum_{c \in \{0,1\}^s} \text{eq}(x, z_x) \text{eq}(c, z_c) L_{\ell+1}(x, 0, c) L_{\ell+1}(x, 1, c)$$

is discharged in two sumcheck phases plus a line step:

1. *Phase A* (in-tree variables; only for $\ell \geq 1$): **[V6]** layer 0 must have `sc_x` absent and layers ≥ 1 present (**Shape**); **[V7]** the phase-A proof's `claimed_sum` must equal the running claim C_ℓ (**LayerClaim**) — this is where the chain from the absorbed roots reaches the layer. Then `MLSumcheck::verify_as_subprotocol(transcript,  $\ell$ , 3, sc_x)` runs an ℓ -round, *degree-3* sumcheck (degree pinned by the verifier, not the proof), yielding a point $r_x \in \mathbb{K}^\ell$ and an expected evaluation E_A . **[V10]** $E_A = \text{eq}(r_x, z_x) \cdot \text{sc.c.claimed_sum}$ (**LayerClaim**) — the factored-eq linkage that hands the residual sum to phase B. (At $\ell = 0$, **[V7]** instead checks `sc.c.claimed_sum` = C_0 directly.)
2. *Phase B* (tree-index variables): `MLSumcheck::verify_as_subprotocol(transcript,  $s$ , 3, sc_c)` yields $r_c \in \mathbb{K}^s$ and E_B . **[V11]** $E_B = \text{eq}(r_c, z_c) \cdot p \cdot q$ where $(p, q) = \text{pair}$ is the layer's closing child pair (**LayerClaim**). The honest values are $p = \widetilde{L}_{\ell+1}(r_x, 0, r_c)$, $q = \widetilde{L}_{\ell+1}(r_x, 1, r_c)$.
3. *Line step*: **A**(0x32: p, q), **S**(λ_ℓ), and the claim advances along the line through the pair:

$$C_{\ell+1} = p + \lambda_\ell (p + q), \quad z_x \leftarrow r_x \parallel \lambda_\ell, \quad z_c \leftarrow r_c.$$

(λ_ℓ is the code's per-layer mu; renamed here to avoid clashing with the residual value μ_l .)

After d layers the exit is $z = (z_x, z_c) \in \mathbb{K}^{d+s}$ with exit value $e_d = C_d$, honestly $\widetilde{\Lambda}(z_x, z_c)$ — pinned by the `merged_forest_roundtrips` test, which checks the exit value equals `mle.at(leaves, z)`.

Inside the generic sumcheck. (`src/piop/sumcheck.rs` + `verifier.rs`.) Both phases use the same subprotocol. It first absorbs the variable count and the degree (via `absorb_random_field`), rejects unless the message count equals the variable count **[V8]**, then per round: absorbs the round message, squeezes the round challenge, and absorbs the challenge back. Round messages carry only the *tail* evaluations $P_i(1), \dots, P_i(\text{deg})$; the verifier reconstructs $P_i(0) := \text{expected}_{i-1} - P_i(1)$, so the per-round sum identity $P_i(0) + P_i(1) = \text{expected}_{i-1}$ holds *by construction* and the protocol's content is the chaining $\text{expected}_i = P_i(r_i)$ (Lagrange interpolation on $\{0, \dots, \text{deg}\}$) together with the caller's use of the final expected evaluation (checks **[V10]**, **[V11]** above; check **[V14]**+division in Section 5). Per-round message length is enforced equal to the degree at the end **[V9]** (`MaxDegreeExceeded`); the absorbs use the messages as sent.

Provenance. In this section: the roots are **(R)** recomputed from proof data and bound by **A**(0x30); C_0 is **(R)** from bound data plus ζ ; the round messages, closing pairs, and `claimed_sum` fields are **(A)** — absorbed, and additionally checked (**[V7]**, **[V10]**, **[V11]**) against already-chained values; all challenges are squeezed; e_d and z are **(R)** functions of bound data and challenges. Nothing here touches the commitment yet: the forest alone binds us to *some* multilinear leaf family; identifying that family with the committed bits is the next section's job.

5 The exit-claim linkage: pre-sumcheck, ring switch, η -batching, one oracle call

This is the load-bearing glue: it turns each chunk’s forest exit claim — an evaluation of the *leaf* MLE, which entangles the committed bits with the public α -power table — into a $\mathbb{K}$ -linear functional of the committed bits alone, and hands the batch of L such functionals to the oracle as *one* inner-product claim.

5.1 From the exit claim to a bit-MLE claim: the pre-sumcheck

By (3) and multilinearity of the extension (the MLE of the all-ones table is 1):

$$e_d^{(l)} - 1 = \sum_{i \in \{0,1\}^d} \sum_{c \in \{0,1\}^s} \text{eq}(i, z_x^{(l)}) \text{eq}(c, z_c^{(l)}) M[c][i] \tau^{(l)}(i) = \sum_{i \in \{0,1\}^d} q_{\text{rowbit}}^{(l)}[i] \cdot m^{(l)}[i], \quad (4)$$

where $q_{\text{rowbit}}^{(l)}[i] = \text{eq}(i, z_x^{(l)}) \tau^{(l)}(i)$ (`row_bit_weights` at $\rho = z_x^{(l)}$) and $m^{(l)}[i] = \sum_c \text{eq}(c, z_c^{(l)}) M[c][i]$ is the z_c -combined row. In $\mathbb{K}$, $e_d - 1 = e_d + 1$; the code checks the $+1$ form.

The pre-sumcheck (`presums[1]`) is a d -round degree-2 sumcheck on the right-hand side of (4), verified by `MultiDegreeSumcheck::verify_as_subprotocol` with one group. The verifier:

- checks `[V14] claimed_sums() = [e_d^{(l)} + 1]` — the sumcheck’s asserted sum is not free data; it must equal the forest-chained exit value, and the slice comparison simultaneously pins the group count to one;
- runs the rounds (shared-challenge machinery, identical absorb/squeeze pattern to Section 4’s subprotocol, with round count pinned to d by the verifier), obtaining the point $r^{*(l)}$ and expected evaluation $E^{(l)}$, honestly $\widehat{R}^{(l)} \cdot \widetilde{m}^{(l)}(r^{*(l)})$;
- recomputes $\widehat{R}^{(l)} = \widetilde{q_{\text{rowbit}}^{(l)}}(r^{*(l)})$ from public data (**R**) — the $O(2^d)$ table build plus an inner product with the $\text{eq}(\cdot, r^{*(l)})$ table — rejects `[V15]` if it is zero (`RHatZero`; for honest runs this happens with probability $\approx 2^{-128}$ over the challenges, *except* in the degenerate statement where every weight of the chunk is zero, which makes $q_{\text{rowbit}}^{(l)} \equiv 0$ and rejects deterministically — a completeness corner, not a soundness issue; an adversary gains nothing by hoping for a zero since rejection is the outcome), and *divides*:

$$\mu_l = E^{(l)} \cdot (\widehat{R}^{(l)})^{-1},$$

which for an honest prover equals $\widetilde{m}^{(l)}(r^{*(l)}) = \widetilde{M}(r^{*(l)}, z_c^{(l)})$ — a pure bit-MLE evaluation at the point $pt_l = (r^{*(l)} \| z_c^{(l)})$, with the α -power weight factor stripped. The division is the “de-black-boxing”: everything α -dependent is now on the verifier’s side of the equation.

Remark 1 (The pre-sumcheck degree rides the proof). *Unlike the forest layers (degree pinned to 3 by the verifier), the pre-sumcheck’s per-group degree is read from the proof, absorbed, and then enforced per round. A malicious prover may declare any degree $\delta < 2^{32}$ (the codec’s `u32`); the sumcheck’s per-round soundness error grows linearly, $\delta/|\mathbb{K}|$, i.e. at most 2^{32-128} per round — negligible. The declared degree is absorbed before any round challenge, so it cannot be chosen adaptively per round. This is a documented asymmetry, not a gap.*

5.2 From L bit-MLE claims to one oracle call: the ring switch

Each $pt_l \in \mathbb{K}^{d+s}$ splits at the packing boundary: $r_{lo}^{(l)} = pt_l[0..7]$ (the seven in-pack row-bit coordinates) and $pt_l^{hi} = pt_l[7..] \in \mathbb{K}^{m_p}$. Writing $\widetilde{M}_v$ for the MLE of the v -th bit-slice of the packed vector P (i.e. $P[y] = \sum_v \widetilde{M}_v\text{-table}[y] \cdot \beta_v$), multilinearity gives $\widetilde{M}(pt_l) = \sum_{v=0}^{127} \text{eq}(v, r_{lo}^{(l)}) \cdot \widetilde{M}_v(pt_l^{hi})$. The prover sends, per chunk, the 128 partial evaluations $s_v^{(l)}$ claimed to be $\widetilde{M}_v(pt_l^{hi})$ (`RingSwitchProof.s_v`); the verifier:

- **[V17]** checks the one $\mathbb{K}$ -linear consistency equation $\sum_v \text{eq}(v, r_{lo}^{(l)}) s_v^{(l)} = \mu_l$ — this ties the message to the forest-chained value μ_l (one constraint out of 128 degrees of freedom; the rest are bound below);
- absorbs each message (`A(0x20)`, all L of them in chunk order), then squeezes $r'' \in \mathbb{K}^7$ and $\eta \in \mathbb{K}^L$;
- recombines each message through the $\mathbb{F}_2$ -linear batching map $\Phi_{r''}: \beta_u \mapsto \text{eq}_{r''}[u]$: with $s_u^{(l)} = \text{transpose}_{128}(s^{(l)})$ (an exact basis reindexing, `transpose_bits_128`),

$$\beta_l = \sum_{u=0}^{127} \text{eq}_{r''}[u] s_u^{(l)}, \quad \beta = \sum_l \eta_l \beta_l \quad (\mathbf{R}).$$

For honest messages, $\beta_l = \sum_y P[y] \cdot B_l(y)$ with

$$B_l(y) = \Phi_{r''}(\text{eq}(pt_l^{hi}, y)\text{-table}[y]),$$

the ring-switch recombination identity (Flock, App. B; ring-switching due to Diamond–Posen). If some $s^{(l)}$ differs from the true partial evaluations, then over the choice of r'' (squeezed *after* the absorb of $s^{(l)}$) the recombined β_l differs from $\sum_y P[y] B_l(y)$ except with probability at most $7 \cdot 2^{-128}$ (Schwartz–Zippel; the difference is a nonzero multilinear polynomial in the 7 coordinates of r''), and over η the batch value β then differs from $\sum_y P[y] B(y)$ except with probability 2^{-128} (the difference is a nonzero linear form in η) — leaving a false (β, B) pair to the oracle, which rejects by Definition 1.

The oracle call. The combined weight function is $B(y) = \sum_l \eta_l B_l(y)$. The verifier never materializes B (that would cost $O(2^{m_p})$); instead it hands the oracle the `EvalB` callback, which evaluates the MLE $\widetilde{B}$ at oracle-chosen points succinctly:

$$\text{EvalB}(\rho, k')[y] = \sum_l \eta_l \cdot \text{residual_b_evals}(\rho, k', pt_l^{hi}, \text{eq}_{r''})[y],$$

the tensor-algebra evaluation of $\widetilde{\Phi_{r''} \circ \text{eq}(pt_l^{hi}, \cdot)}$ at $\rho \parallel \text{bits}(y)$ in $O((|\rho| + 2^{k'}) \cdot 128^2)$ $\mathbb{K}$ -operations, with no 2^{m_p} -sized table (`residual_b_evals / tensor_eq_phi_eval`, `src/ligerito.rs`; pinned against brute force by tests). The callback is verifier-side code computing a public function of (pt_l^{hi}, r'', η) and the oracle’s own challenges; *when* the oracle invokes it is inside the box. The oracle call **[V18]** then certifies $\sum_y P[y] B(y) = \beta$ against the committed root — the only step in the entire verification that touches the commitment.

Quantity	Class	Bound / checked by
c_w, L , chunk weights $W^{(l)}$, bound 2^{127}	(R)	statement (p, q_{bits}, w)
<code>us[1]</code> (sent folds)	(A)	[V1] range; absorbed as roots α^u (tag 0x30); [V19]
roots $R_c^{(l)}$	(R)	deterministic in <code>us[1]</code> , α ; the absorb binds
$\zeta^{(l)}, \lambda_\ell$, round challenges, $r^{*(l)}, r'', \eta$	—	squeezed (uniform over $\mathbb{K}$)
entry claim $C_0^{(l)} = \tilde{R}(\zeta)$	(R)	<code>mle.at</code> on bound roots
sumcheck round messages (all layers, pre-sum)	(A)	absorbed; length/degree checks [V8][V9][V12][V13]
<code>claimed_sum</code> fields (<code>sc.x</code> , <code>sc.c</code>)	(A)	[V7] = running claim; [V10] phase linkage
closing pairs (p, q)	(A)	absorbed (tag 0x32); [V11] against E_B
exit $(z^{(l)}, e_d^{(l)})$	(R)	chained from bound data + challenges
presum <code>claimed_sums</code>	(A)	[V14] = $[e_d^{(l)} + 1]$
presum declared degree	(A)	absorbed; per-round length [V13] (Remark 1)
$\hat{R}^{(l)}, \mu_l, pt_l$	(R)	public table + division; [V15]
<code>s_v</code>	(A)	[V17] one linear check vs. μ_l ; absorbed (tag 0x20)
β_l, β	(R)	deterministic in bound <code>s_v</code> , r'', η
EvalB values	(R)	public function of points, r'', η , oracle challenges
$\sum_y P[y]B(y) = \beta$	(O)	[V18], Definition 1
y_{ver}	(R)	from <code>us</code> , w' ; [V19] vs. <code>claimed</code>
vc, m_p	(R)	shape-derived (Section 7)

Table 1: Provenance of verifier-side quantities. (R) recomputed from public data and verifier randomness; (A) prover data bound by an absorb and/or an equality check against bound data; (O) oracle-certified.

5.3 Provenance of every verifier-side quantity

Reading the chain backwards from the oracle: [V18] certifies the committed bits enter β ; β deterministically encodes the absorbed `s_v` under post-absorb randomness (r'', η) ; [V17] ties `s_v` to μ_l ; μ_l is a public-data quotient of the pre-sumcheck’s expected evaluation, whose asserted sum [V14] is the forest exit value; the forest chains that value to the absorbed roots under fresh challenges; the roots are α^{us} ; and [V1] + the generator property make the exponent map injective there, so `us` must be the true folds of the committed bits. [V19] then evaluates the statement on `us`. Each arrow is one of the checks above; none is “the verifier recomputed a value the prover chose freely.”

6 Integer binding in the clear

This section is the $\mathbb{K}$ -side trust anchor: it is what lets the verifier treat the *sent integers* `us` as the true folds once the forest and the oracle have done their work.

The generator check. [V3] `is_generator( $\alpha$ ) (src/pcs.rs)`: reject unless $\alpha \notin \{0, 1\}$ and $\alpha^{(2^{128}-1)/p} \neq 1$ for every prime $p \mid 2^{128} - 1$, using the hard-coded squarefree factorization

$$2^{128} - 1 = 3 \cdot 5 \cdot 17 \cdot 257 \cdot 641 \cdot 65537 \cdot 274177 \cdot 6700417 \cdot 67280421310721$$

(GF128_ORDER_PRIME_FACTORS: the five Fermat primes dividing $2^{32} - 1$, then $2^{32} + 1 = 641 \cdot 6700417$ and $2^{64} + 1 = 274177 \cdot 67280421310721$). This is the standard primitive-element test; it passes iff $\text{ord}(\alpha) = 2^{128} - 1 = \text{GF128_MULT_ORDER}$ (`= u128::MAX`). *Why it is needed:* the exponent map $n \mapsto \alpha^n$ is injective on $[0, \text{ord}(\alpha))$; since $2^{128} - 1$ is composite, a non-generator α could have order

as small as 3, collapsing the binding entirely. With a generator, the map is injective on all of $[0, 2^{128} - 1)$, which strictly contains the range-checked domain $[0, 2^{127})$. About $\prod_p (1 - 1/p) \approx 49\%$ of $\mathbb{K}^\times$ are generators, so a randomly sampled α passes after ≈ 2 resamples; the verifier simply rejects a non-generator. The check is repeated (identically) per chunk; it depends only on α .

The range check. [V1], in `verify_mle_eval_mod_q_ligerito` itself: for every chunk l and column c , reject `ChunkRange{chunk, col}` unless

$$u_c^{(l)} < 2^{c_w+t+W} = 2^{127}$$

(the exponent $c_w + t + W = 127$ by construction of c_w ; the code computes the bound generically as `1u128 << (c_w + t + word_bits)`). The bound is exactly the honest-fold envelope (2): honest chunk folds always pass, and any sent fold that passes lies in the injective range. The check runs *before* the chunk’s transcript activity and covers every entry of `us[1]`. (The inner [V4] magnitude guard, $\max u < 2^{128} - 1$, is inherited from the unchunked path and is strictly weaker; it is vacuous given [V1].)

The binding argument. Fix a chunk l and column c . Let $u^{\text{true}} = \sum_b W_b^{(l)} \text{INT}(D(b, c))$ be the fold of the *committed* bits. The forest + pre-sumcheck + ring switch + oracle chain (Sections 4–5) forces — up to the stated soundness errors — the per-tree product of the leaf family (3) built from the committed bits to equal the absorbed root, i.e. $\alpha^{u^{\text{true}}} = \alpha^{u_c^{(l)}}$. Both exponents lie in $[0, 2^{127})$: the left by (2), the right by [V1]. Injectivity of $n \mapsto \alpha^n$ on $[0, 2^{128} - 1) \supset [0, 2^{127})$ (the generator property, [V3]) gives $u_c^{(l)} = u^{\text{true}}$. Both checks are load-bearing: without [V3] the map need not be injective; without [V1] the prover could send $u + \text{ord}(\alpha) \cdot k$. Note the argument needs no randomness in α — any generator fixed before the prover commits to `us` works (cf. Obligation 1).

Root recomputation. $R_c^{(l)} = \alpha^{u_c^{(l)}}$ is computed with an 8-bit-window fixed-base comb — `FixedBasePow::new` at $(\alpha, 128, 8)$ — a pure optimization of `gf_pow`: 16 table lookups/multiplies per exponent instead of a 128-step square-and-multiply chain.

The mod- q recombination. [V19] (`recombine_read_off, src/pcs.rs`): in the evaluation ring R ,

$$y_{\text{ver}} = \sum_{c \in \{0,1\}^s} w'_c \cdot \sum_{l=0}^{L-1} (2^{c_w})^l \cdot u_c^{(l)},$$

computed with 2^{c_w} lifted once ($c_w \leq 126$, so it is an exact `u128`) and powered in R ; the flattened index is `v_flat[l · 2s + c] = us[1][c]`. The wide value $\sum_l 2^{c_w l} u_c^{(l)}$ never materializes as an integer — the whole point of chunking — and the column weights enter at full width, unchunked. Reject `ReadOff` unless $y_{\text{ver}} = \text{claimed}$. Correctness of the recombination against (1) is the telescoping $\sum_l 2^{c_w l} W_b^{(l)} = w_b$ (exact under the width precondition) and the ring homomorphism $R::\text{from}: [0, 2^{127}) \rightarrow R$ being faithful on the values that occur — a statement-side precondition on R (for $R = \mathbb{F}_q$ with q a 100-bit prime, reduction mod q is the canonical map; “faithful” means the canonical composite, not injectivity).

7 Config derivation and the audited regime

The Ligerito verifier config *vc* is *derived from the instance shape, never from the proof*: `sha_lig_configs( $m_p$ )` (`src/ligerito_flock.rs`) selects, with $m = m_p + 7$:

- $m \geq 22$: the *embedded FAST profile* at m — the *audited* regime;
- $m < 22$: `ligerito::default_config` at $(m_p, \text{log_batch} = 2, \text{log_inv_rate} = 2)$ — ad-hoc, *unaudited*, test-only.

The embedded profiles are flock’s own audited security configs (deserialized from embedded TOML via `LigeritoSecurityConfig`, existing for $m = 22..35$); they also dictate the *commit* shape (`log_inv_rate = log_inv_rates[0]`, `log_batch = initial_k`), so prover, committer, and verifier all derive the same parameters from m_p alone. The ad-hoc branch builds unique-decoding-regime query counts with no grinding and no out-of-domain samples, and its per-level parameters are not audited.

Scope of this note’s claims. Everything this note asserts about the oracle (Definition 1) is asserted *for the embedded-profile regime $m \geq 22$ only* — every deployed shape in the README’s reference table from $n = 22$ up. Runs below that boundary (including the repository’s own unit tests, Section 8) use the unaudited ad-hoc config: the verification *logic* is identical, but the oracle’s concrete soundness there carries no audit and is outside this note’s claims. A caller may also bypass `sha_lig_configs` and pass any `LigConfig` to `lig_configs`; the same scoping applies. (Performance is not this note’s subject, but the boundary is visible in the README’s tables as the $n = 20 \rightarrow 22$ step in proof size; hard-coding the ad-hoc config at large shapes is also catastrophically slow — measured 292s vs. tens of ms for an $n = 28$ commit.)

8 What the tests pin

The suite (`cargo test --release`: 68 lib tests, of which one is an ignored measurement baseline, plus 2 doctests) pins the verifier’s behavior at these points; test names are exact.

- `mle_eval_mod_q_ligerito_roundtrips` (`src/ligerito_flock.rs`) — the end-to-end reference, at $(t, s, W) = (10, 5, 1)$ ($L = 1$) and $(4, 8, 32)$ ($L = 2$), with a concrete $\mathbb{F}_q$, $q = 2^{100} - 15$. Arms: honest proof accepts; wrong claimed y rejects with `ReadOff` [V19]; a tampered chunk fold (`us[0][0]  $\leftarrow 2^{128} - 2$`) rejects with `ChunkRange` [V1]; a non-generator $\alpha = 1$ rejects with `ChallengeNotGenerator` [V3].
- `mod_q_ligerito_proof_serialization_roundtrips` — decode $\circ$ encode preserves acceptance; re-serialization is byte-identical (canonicity of `to_bytes`); flipping a single byte at each of four positions $(0, \lfloor n/3 \rfloor, \lfloor n/2 \rfloor, n - 1)$ is rejected — the stream fails to decode or the decoded proof fails verification.
- `merged_forest_roundtrips` (`src/merged_forest.rs`) — prover/verifier exit-claim agreement over four (d, s) shapes; roots equal the per-tree leaf products; *the exit value equals `mle_at(leaves, z)`* (the semantic anchor for Section 4); tamper arms: a phase-A `claimed_sum` [V7], a layer-0 closing pair [V11], and the last layer’s phase-B `claimed_sum` each reject.
- `lazy_matches_eager` and `lazy_multi_matches_eager` — both lazy prover schedules ($L/4$ default and `F2.FOREST_SCHEDULE=18`) are byte-identical to the eager reference prover: same roots, same per-layer messages and pairs, same exit claim, *same post-run transcript state* (a squeezed challenge compares equal). This is what lets the verifier be schedule-agnostic.

- `tensor_eval_matches_bruteforce_and_table` and `residual_b_evals_matches_pointwise` (`src/ligerito.rs`) — the succinct EvalB algebra equals the brute-force $\sum_y \text{eq}(y, \cdot) \Phi_{r''}(\cdot)$ and the per-boolean-tail pointwise evaluations.
- `field_bridge_is_bit_identical` (`src/ligerito_flock.rs`) — the zinc/flock $\mathbb{K}$ representations agree as raw words under $+$ and $\times$, and `LOG_PACKING` equals flock’s `pcs::pack::LOG_PACKING`. `neon_mul_matches_scalar_pipeline` pins the aarch64 NEON field pipeline bit-identical to the portable scalar one.
- `packed_cols_row_roundtrip` and the `poly/` field-axiom and `lift/`projection tests — layout inversions and $\mathbb{K}$ arithmetic.

What the tests do *not* cover. Honest gaps a reviewer should know:

- *Fiat–Shamir order mutations.* No test mutates an absorb order, a domain tag, or a framing byte and asserts rejection; the schedule of Section 3 is pinned only indirectly (honest-roundtrip acceptance requires prover/verifier lockstep, and the byte-identity tests pin the two prover variants to each other). A transcript-schedule regression that changed prover and verifier *together* would pass the suite.
- *Codec field coverage.* The tamper test flips four fixed byte positions of one proof; there is no exhaustive per-field tamper sweep, and no test distinguishes clean `CodecError` returns from the panic paths of Section 2.4.
- *Rejection-arm coverage.* `Shape`, `RingSwitchClaim` [V17], `PreSumcheck` [V14], `RHatZero` [V15], and `LigeritoReject` [V18] have no dedicated negative tests in this repository (some are hit probabilistically by the byte-flip test).
- *The audited regime is not unit-tested.* All `cargo test` shapes sit below the $m \geq 22$ boundary and run the ad-hoc config; the embedded profiles are exercised only by the `reference_measure` example and the bench suite (Section 7).
- *Batched and virtual-XOR variants* have no tests in this repository (they were dropped in the extraction; the code is a verbatim port).

9 Trust boundary and caveats

The oracle assumption. Everything rests on Definition 1 holding for `flock-core`’s recursive Ligerito verifier under the shape-derived config (Section 7). `flock-core` is consumed as a *local path dependency* (`flock-core = { path = ".../flock/crates/flock-core" }` in `Cargo.toml`, mirroring the upstream pin; it pins `bincode 1.3` and `serde 1`; MIT OR Apache-2.0). There is no version or content pin beyond the checkout itself — reproducing a verification requires the same flock tree. The oracle’s internal Fiat–Shamir sub-schedule runs over the shared transcript through `ZincChallenger` (Section 3.1) and is trusted along with the rest of the box.

Known upstream item: Merkle domain separation. `flock`’s `merkle` module does not domain-separate leaf hashing from internal-node hashing, and its own module note flags it as a “micro-benchmark module, not production code.” F2Z inherits flock’s commitment and Merkle verbatim; until flock ships the fix, the commitment binding — and therefore Definition 1 — should be treated as pre-production. (Recorded in `README.md` and `docs/DESIGN.md`; this note adds nothing beyond scoping its claims accordingly.)

Caller obligations (summary). Obligation 1 (statement/root absorbed before challenges; α fixed pre-commitment); weight vectors of the right lengths ($2^t, 2^s$) with $w_b < 2^{q_{\text{bits}}}$; q_{bits} consistent with the intended $\mathbb{F}_q$; R of characteristic $\neq 2$ with `From<u128>` the canonical map (a characteristic-2 R would collapse the read-off to parity — the very obstruction the exponent fold exists to avoid); the commitment produced at the same shape `p` handed to the verifier. The proof-stream decoder is not hardened against hostile bytes (Section 2.4).

Not zero-knowledge. The Ligerito opening reveals queried committed data; no masking exists anywhere on this path.

Out-of-scope public API. The batched multi-polynomial opener (`verify_rs_ligerito_batch`) and the virtual-XOR variants (`verify_mle_eval_mod_q_ligerito_with_virtual_xors / ..._claims_only`) share this note’s machinery but add statement-level structure this note does not review. One contractual point deserves flagging even here: the virtual-XOR verifiers *return* a `Vec<VirtualXorObligation>` — external-term residual claims $\hat{e}(pt)$ = value that ride the proof *unverified* and that the *caller* must discharge by other means. A host that ignores the returned obligations has not verified those claims. These variants also carry no tests in this repository (Section 8).

What a correctness argument for F2Z still owes. Beyond this note’s line-by-line checkable description: (i) the Ligerito oracle itself (proximity soundness, Merkle binding incl. the domain-separation item, its FS sub-schedule) in the embedded-profile regime — and separately anything run under the ad-hoc config; (ii) the host-level statement binding of Obligation 1; (iii) quantitative composition of the stated per-step soundness errors (forest layers, pre-sumcheck, ring-switch recombination, η -batching) into one bound — this note states each check’s role and error shape but proves nothing; (iv) hardening of `from_bytes` for hostile inputs.

Verifier cost. For the record (README reference table, Apple M4): verification is milliseconds-class across the deployed range — 1.2–14.8 ms for $n = 16$ –32 at $t \approx 0.6n$, $W = 1$, one mod- q claim — with proofs 43–574 KiB. The verifier’s only super-logarithmic work is the per-chunk $O(2^d) q_{\text{rowbit}}$ table (dominated by 2^t fixed-base exponentiations) and the $O(2^s)$ root/entry/read-off loops.

References

- [1] L. Soukhanov. *Characteristic-2 field switching* (the char2-fieldswitch note), §9: a one-round, characteristic-2 Brakedown instantiation. The construction F2Z implements.
- [2] B. Bünz, J. Rothblum, R. Wang (Succinct Labs). *Flock: Fast Proving for Batch Boolean Computations*. Source of the ring-switch implementation shape (App. B) and of `flock-core`, this crate’s opener backend.
- [3] B. E. Diamond, J. Posen. *Polylogarithmic Proofs for Multilinears over Binary Towers* (ring-switching). The packing/recombination technique behind Section 5.
- [4] A. Novakovic, G. Angeris. *Ligerito: A Small and Concretely Fast Polynomial Commitment Scheme*. The recursive opener behind Definition 1.
- [5] The `zinc-plus` repository, `documentation/f2-int-eval-doc/` (branch `f2-int-sha`): the as-deployed long-form exposition this note’s notation follows. Where the deployment and this repository disagree, this repository’s code is authoritative here.